

C# BASICS | ArrowEscaper

クラスの継承

ゲームを作りながら、継承のありがたみを実感する

対象：1年生（構造体・クラス既習）

INHERIT

LESSON 1

Player ・ Arrow を作る

配布したプロジェクトに、動くキャラクターを追加する



このコマの目標

- MonoBehaviour を継承して、Player クラスを作れるようになる
- 同じく MonoBehaviour を継承して、落ちてくる Arrow クラスを作れるようになる
- Arrow を一定間隔で生み出す ArrowSpawner クラスを作れるようになる
- DemoScene の Hierarchy に登録し、実際に動かして確認する

Player クラスの設計

- MonoBehaviour を継承し、Update ・ Draw を override する
- x, y 座標と、hp（体力）を持たせる
- Update では、WASD キー入力を読み取って座標を動かす
- Draw では、自分自身と、HP などの UI 表示を行う

Player クラスの実装①: フィールドと Update

c# Player.cs

```
1 public class Player : MonoBehaviour
2 {
3     public int x = 14;
4     public int y = 14;
5     private int hp = 3;
6
7     public override void Update()
8     {
9         if (Input.GetKey(KeyCode.W)) { y--; }
10        if (Input.GetKey(KeyCode.S)) { y++; }
11        if (Input.GetKey(KeyCode.A)) { x--; }
12        if (Input.GetKey(KeyCode.D)) { x++; }
13
14        x = Math.Clamp(x, 0, Renderer.Width - 1);
15        y = Math.Clamp(y, 0, Renderer.Height - 1);
16    }
```

Player クラスの実装②: Draw ・ Damage

- Damage は MonoBehaviour には無い、Player 独自のメソッド。HP が 0 になったらリザルト画面へ切り替える。

c# Player.cs (続き)

```
1 public override void Draw()
2 {
3     Renderer.Draw(x, y, 'P');
4     Renderer.DrawUI(0, "HP : " + hp);
5 }
6
7 public void Damage(int damage)
8 {
9     hp -= damage;
10
11     if (hp <= 0)
12     {
13         hp = 0;
14         SceneManager.LoadScene(new ResultScene());
15     }
16 }
```

Arrow クラスの設計

- 同じく MonoBehaviour を継承する。x 座標はコンストラクタで受け取り、y は 0 からスタート
- Start で、FindObjectOfType<Player>() を使って Player を探しておく
- Update で一定間隔ごとに下へ移動し、Player と同じ座標になったらダメージを与える

POINT

FindObjectOfType は MonoBehaviour 側に用意されている、継承したクラスなら誰でも使える機能でしたね。

Arrow クラスの実装①: コンストラクタと Start

c# Arrow.cs

```
1 public class Arrow : MonoBehaviour
2 {
3     public int x;
4     public int y;
5
6     private int moveInterval = 3;
7     private int moveIntervalCounter = 0;
8
9     private Player player;
10
11     public Arrow(int x)
12     {
13         this.x = x;
14         y = 0;
15     }
16
17     public override void Start()
18     {
19         player = FindObjectOfType<Player>();
20     }
```


Arrow クラスの実装②: Update ・ Move ・ Draw

c# Arrow.cs (続き)

```
1 public override void Update()
2 {
3     moveIntervalCounter++;
4     if (moveIntervalCounter >= moveInterval)
5     {
6         Move();
7         moveIntervalCounter = 0;
8     }
9 }
10
11 private void Move()
12 {
13     y++;
14     if (player != null && x == player.x && y == player.y)
15     {
16         player.Damage(1);
17     }
18 }
19
20 public override void Draw()
21 {
22     Renderer.Draw(x, y, 'V');
23 }
```

ArrowSpawner クラスの実装

c# ArrowSpawner.cs

```
1 public class ArrowSpawner : MonoBehaviour
2 {
3     private Random random = new Random();
4     private int spawnFrame = 0;
5     private int spawnFrameCounter = 30;
6
7     public override void Update()
8     {
9         spawnFrame++;
10        if (spawnFrame >= spawnFrameCounter)
11        {
12            SpawnArrow();
13            spawnFrame = 0;
14        }
15    }
16
17    private void SpawnArrow()
18    {
19        int x = random.Next(0, Renderer.Width);
20        Instantiate(new Arrow(x));
21    }
22 }
```

DemoScene への登録

- 配布時は空だった Hierarchy に、作った 3 つのクラスと GridBackground を追加する。

c# DemoScene.cs

```
1 public class DemoScene : Scene
2 {
3     public override void Hierarchy()
4     {
5         Add(new GridBackground());
6         Add(new ArrowSpawner());
7         Add(new Player());
8     }
9 }
```

POINT

GridBackground は配布済み。Add した順番が Draw ・ Update される順番にもなります。

このコマのまとめ

- MonoBehaviour を継承し、Update ・ Draw を override することでキャラクターを 1 つ作れた
- Arrow はコンストラクタで初期値を受け取り、FindObjectOfType で他のオブジェクトを参照できた
- Hierarchy に登録するだけで、Scene や MainSystem の仕組みに乗って自動的に動き出す

LESSON 2

シーン遷移をつくる

タイトル→ゲーム→リザルトの流れを完成させる



このコマの目標

- TitleScene ・ ResultScene を作り、 SceneManager で画面を切り替えられるようにする
- 生き残った時間を得点として表示する ScoreManager を作る
- タイトル→ゲーム→リザルト→タイトル、と一巡できる状態を完成させる

シーン遷移の仕組み（復習）

- `SceneManager.LoadScene(new ooScene())` で、次に切り替えるシーンを予約する
- 実際の切り替えは `MainSystem` が毎フレーム呼ぶ `ChangeScene()` の中で行われる
- 切り替わると、新しい `Scene` の `Hierarchy()` が呼ばれ、中の登場人物が配置される

POINT

Player の Damage から、すでに `SceneManager.LoadScene(new ResultScene())` を呼んでいましたね。

TitleScene ・ TitleObject の実装

c# Program.cs

```
1 public class TitleObject : MonoBehaviour
2 {
3     public override void Update()
4     {
5         if (Input.GetKeyDown(KeyCode.Enter))
6         {
7             SceneManager.LoadScene(new DemoScene());
8         }
9     }
10
11     public override void Draw()
12     {
13         Renderer.Draw(9, 5, "ArrowEscaper");
14         Renderer.Draw(10, 9, "Press Enter");
15     }
16 }
17
18 public class TitleScene : Scene
19 {
20     public override void Hierarchy()
21     {
22         Add(new TitleObject());
23     }
24 }
```


ResultScene ・ ResultObject の実装

- ScoreManager.Score を表示し、 Enter キーでタイトルへ戻るだけのシンプルな作り。

c# ResultObject.cs

```
1 public class ResultObject : MonoBehaviour
2 {
3     public override void Update()
4     {
5         if (Input.GetKeyDown(KeyCode.Enter))
6         {
7             SceneManager.LoadScene(new TitleScene());
8         }
9     }
10
11     public override void Draw()
12     {
13         Renderer.Draw(12, 4, "RESULT");
14         Renderer.Draw(7, 7, "SCORE : " + ScoreManager.Score.ToString("F1"));
15     }
16 }
```

ScoreManager の実装

c# ScoreManager.cs

```
1 public class ScoreManager : MonoBehaviour
2 {
3     public static double Score { get; private set; }
4     private int survivalFrame = 0;
5
6     public override void Start()
7     {
8         Score = 0;
9         survivalFrame = 0;
10    }
11
12    public override void Update()
13    {
14        survivalFrame++;
15        Score = (double)survivalFrame / MainSystem.FPS;
16    }
17
18    public override void Draw()
19    {
20        Renderer.DrawUI(2, "TIME : " + Score.ToString("F1"));
21    }
22 }
```

一連の流れを確認する

- DemoScene.Hierarchy に ScoreManager も追加しておく
- ① タイトル画面で Enter → ② ゲーム開始、矢をよけ続ける → ③ HP が 0 でリザルト画面 → ④ Enter でタイトルへ戻る
- この一巡が問題なく動けば、今日のゴールは達成です

このコマのまとめ

- TitleScene ・ ResultScene も、 DemoScene と同じく Scene を継承して作れた
- SceneManager.LoadScene(...) で、いつでも好きなタイミングで画面を切り替えられる
- これで「タイトル→ゲーム→リザルト」が一巡する、遊べる状態が完成した

LESSON 3

アイテムを追加する

回復アイテムとスコアアップアイテムを作る

3

このコマの目標

- 触れると HP が回復する Heart クラスを作る
- 触れるとスコアが増える ScoreItem クラスを作る
- 2 つとも、Arrow と同じような「Player との当たり判定」の作りになることを確認する

Heart クラスの実装①：構築と Start

c# Heart.cs

```
1 public class Heart : MonoBehaviour
2 {
3     public int x;
4     public int y;
5     private Player player;
6
7     public Heart(int x, int y)
8     {
9         this.x = x;
10        this.y = y;
11    }
12
13    public override void Start()
14    {
15        player = FindObjectOfType<Player>();
16    }
```

Heart クラスの実装②: Update ・ Draw

- Arrow と同じ形の当たり判定。触れたら Heal を呼び、enabled を false にして消す。

C# Heart.cs (続き)

```
1 public override void Update()
2 {
3     if (player != null && x == player.x && y == player.y)
4     {
5         player.Heal(1);
6         enabled = false;
7     }
8 }
9
10 public override void Draw()
11 {
12     Renderer.Draw(x, y, 'H');
13 }
```


Player への追加: Heal

C# Player.cs (追記)

```
1 public void Heal(int amount)
2 {
3     hp += amount;
4 }
```

ScoreManager の拡張: AddScore

- アイテムを取ったぶんのボーナス点を、生存時間のスコアに上乗せできるようにする。

```
C# ScoreManager.cs  
(変更)  
  
1 private static double bonusScore = 0;  
2  
3 public static void AddScore(double amount)  
4 {  
5     bonusScore += amount;  
6 }  
7  
8 // Update の中を書き換え  
9 Score = (double)survivalFrame / MainSystem.FPS + bonusScore;
```

ScoreItem クラスの実装①：構築と Start

- Heart とほとんど同じ形。

c# ScoreItem.cs

```
1 public class ScoreItem : MonoBehaviour
2 {
3     public int x;
4     public int y;
5     private Player player;
6
7     public ScoreItem(int x, int y)
8     {
9         this.x = x;
10        this.y = y;
11    }
12
13    public override void Start()
14    {
15        player = FindObjectOfType<Player>();
16    }
```

ScoreItem クラスの実装②: Update ・ Draw

- 違うのは、当たったときに呼ぶ処理（ScoreManager.AddScore）と、描画する文字だけ。

C# ScoreItem.cs (続き)

```
1 public override void Update()
2 {
3     if (player != null && x == player.x && y == player.y)
4     {
5         ScoreManager.AddScore(10);
6         enabled = false;
7     }
8 }
9
10 public override void Draw()
11 {
12     Renderer.Draw(x, y, 'S');
13 }
```

DemoScene への追加と動作確認

C# DemoScene.cs
(変更)

```
1 public override void Hierarchy()  
2 {  
3     Add(new GridBackground());  
4     Add(new ScoreManager());  
5     Add(new ArrowSpawner());  
6     Add(new Player());  
7     Add(new Heart(5, 5));  
8     Add(new ScoreItem(20, 10));  
9 }
```

POINT

Heart と ScoreItem が表示され、Player が重なると効果が出て消えることを確認しましょう。

このコマのまとめ

- Heart ・ ScoreItem を追加し、遊びの幅が広がった
- ただし、 Arrow ・ Heart ・ ScoreItem の 3 つを見比べると、とてもよく似たコードが並んでいる
- x, y, player, Start, 「 Player と重なったか」の判定...同じ部分が 3 回書かれている状態

POINT

次回は、この「そっくりな部分」を 1 つにまとめる作業をします。

LESSON 4

共通処理を親クラスにまとめる

重複コードを、継承でスッキリさせる



このコマの目標

- Arrow ・ Heart ・ ScoreItem に共通する部分を見つけ出す
- その共通部分を FieldObject という新しい親クラスにまとめる
- Arrow ・ Heart ・ ScoreItem を、FieldObject を継承する形に書き換える
- 書き換えた後も、今までと同じように動くことを確認する

今のコードの重複に気づく

- Arrow ・ Heart ・ ScoreItem 、 3 つとも次の部分がそっくりそのまま重複しています。
- `public int x; public int y; private Player player;`
- コンストラクタで `x, y` を受け取る部分
- `Start()` で `player = FindObjectOfType<Player>();` とする部分
- 「`player != null && x == player.x && y == player.y`」という当たり判定の条件

注意

同じコードが 3 か所にあると、直すときも 3 か所すべてを直す必要が出てきます。

共通の親クラス FieldObject の設計

- MonoBehaviour を継承した、新しい中間の親クラス FieldObject を作る
- x, y, player は、MonoBehaviour ではなく FieldObject に持たせる
- コンストラクタと Start() も、FieldObject にまとめて書いてしまう
- 「Player と重なっているか」を判定する IsTouchingPlayer() を用意し、子クラスから使えるようにする

POINT

Arrow は MonoBehaviour ではなく FieldObject を継承する形に変わります。継承は何段でも重ねられます。

FieldObject クラスの実装

c# FieldObject.cs

```
1 public class FieldObject : MonoBehaviour
2 {
3     public int x;
4     public int y;
5     protected Player player;
6
7     public FieldObject(int x, int y)
8     {
9         this.x = x;
10        this.y = y;
11    }
12
13    public override void Start()
14    {
15        player = FindObjectOfType<Player>();
16    }
17
18    protected bool IsTouchingPlayer()
19    {
20        return player != null && x == player.x && y == player.y;
```

POINT

protected は「自分と、継承した子クラスからだけアクセスできる」印。 public より少しだけ制限が強い。

Arrow の書き換え①：継承元とコンストラクタ

- MonoBehaviour を FieldObject に変え、共通部分をごっそり削除する。base(x, 0) で親のコンストラクタを呼ぶ。

C# Arrow.cs (書き換え後)

```
1 public class Arrow : FieldObject
2 {
3     private int moveInterval = 3;
4     private int moveIntervalCounter = 0;
5
6     public Arrow(int x) : base(x, 0) { }
```

Arrow の書き換え②: Update ・ Move ・ Draw

C# Arrow.cs (書き換え後・続き)

```
1  public override void Update()
2  {
3      moveIntervalCounter++;
4      if (moveIntervalCounter >= moveInterval)
5      {
6          Move();
7          moveIntervalCounter = 0;
8      }
9  }
10
11 private void Move()
12 {
13     y++;
14     if (IsTouchingPlayer())
15     {
16         player.Damage(1);
17     }
18 }
19
20 public override void Draw()
21 {
22     Renderer.Draw(x, y, 'V');
23 }
24 }
```

Heart ・ ScoreItem の書き換え

- 同じように FieldObject を継承すると、どちらも Update と Draw だけの短いクラスになる。

C# Heart.cs (書き換え後)

```
1 public class Heart : FieldObject
2 {
3     public Heart(int x, int y) : base(x, y) { }
4
5     public override void Update()
6     {
7         if (IsTouchingPlayer())
8         {
9             player.Heal(1);
10            enabled = false;
11        }
12    }
13
14    public override void Draw()
15    {
16        Renderer.Draw(x, y, 'H');
17    }
18 }
```

ScoreItem の書き換え・動作確認

C# ScoreItem.cs (書き換え後)

```
1 public class ScoreItem : FieldObject
2 {
3     public ScoreItem(int x, int y) : base(x, y) { }
4
5     public override void Update()
6     {
7         if (IsTouchingPlayer())
8         {
9             ScoreManager.AddScore(10);
10            enabled = false;
11        }
12    }
13
14    public override void Draw()
15    {
16        Renderer.Draw(x, y, 'S');
17    }
18 }
```

POINT

書き換えたら、今まで通り Arrow に当たる・Heart/ScoreItem を拾えることを必ず確認しましょう。

このコマ・全体のまとめ

- Arrow ・ Heart ・ ScoreItem の重複コードを、FieldObject という親クラスにまとめた
- 書き換え後も見た目の動きはまったく変わらないが、コードの量と重複は大きく減った
- 「新しいアイテムを増やす」のも、FieldObject を継承するだけで済むようになった
- これが、継承をあとから使って設計を整理する＝リファクタリングの効果です

お疲れさまでした

MonoBehaviour → FieldObject → Arrow/Heart/ScoreItem という
2段階の継承を、実際に自分の手で作り上げました。